

Reporte Técnico de Actividades Práctico- Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante	Alexander Ventura Guaman Puli
Asignatura	Matemáticas Discretas
Ciclo	1er Ciclo
Unidad	Unidad 1: Lógica Matemática
Resultado de aprendizaje de la unidad	Desarrolla habilidades de razonamiento lógico para solucionar problemas ligados a la ingeniería, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001 Fase 2
Título de la Práctica	Proposiciones, Conectores Lógicos, Inferencia, Deducción y Tablas de Verdad.
Nombre del Docente	Mario Enrique Cueva Hurtado
Fecha	06 de abril al 15 de mayo de 2026
Horario	Lunes, martes, jueves 07:30 – 09:30
Lugar	Aula de la Carrera de Computación / Aula Virtual (Zoom)
Tiempo planificado en el Sílabo	18 horas (3 horas por sesión APE)

2. Objetivos de la Práctica

- Identificar y clasificar proposiciones lógicas simples y compuestas, distinguiendo entre proposiciones y enunciados no proposicionales.
- Comprender y aplicar los conectores lógicos fundamentales (negación, conjunción, disyunción, condicional, bicondicional) en la formulación de expresiones lógicas.
- Construir tablas de verdad para proposiciones compuestas, determinando su valor de verdad en todos los casos posibles.
- Aplicar las leyes de inferencia y deducción lógica para resolver problemas de razonamiento lógico en contextos computacionales y de ingeniería.
- Desarrollar habilidades de trabajo colaborativo, pensamiento crítico y resolución de problemas, integrando principios de equidad, inclusión y responsabilidad ética en el aprendizaje.

3. Materiales, Reactivos, Equipos y Herramientas

- Pizarra y marcadores
- Proyector multimedia



unl

Universidad
Nacional
de Loja

- Hojas de trabajo impresas con ejercicios de lógica proposicional y tablas de verdad
- Guía de ejercicios de inferencia y deducción lógica
- Calculadora científica
- Material bibliográfico de apoyo (textos digitalizados de referencia)
- PC o laptop (mínimo 1 por estudiante)
- Plataforma EVA de la UNL para entrega de evidencias
- Entorno virtual de aprendizaje (EVA-UNL)
- Herramientas colaborativas: Google Drive, Wiki
- Software de presentación (PowerPoint, Canva)
- Máximo de personas por grupo: 4-5 estudiantes

4. Procedimiento / Metodología Ejecutada

Primero se comenzó repasando lo básico de la lógica proposicional, viendo qué son las proposiciones simples y compuestas, y cómo funcionan los conectores lógicos como la negación, conjunción, disyunción, condicional y bicondicional. Después, los estudiantes se organizaron en grupos para trabajar juntos, discutir ideas y resolver los ejercicios.

Luego, se hicieron tablas de verdad con dos, tres y hasta cuatro variables, siguiendo un orden claro para encontrar todos los posibles resultados. Esto ayudó a entender mejor cómo funcionan los conectores y el orden en que se deben aplicar.

Más adelante, se trabajó en simplificar expresiones lógicas usando leyes como las de Morgan, la distributiva, identidad y complemento, con el fin de hacerlas más fáciles de entender.

También se relacionó todo esto con la programación, viendo cómo estas ideas se aplican en estructuras como if-else y while, que son clave para tomar decisiones en los programas.

Al final, todo lo trabajado (tablas, ejercicios y análisis) se organizó en portafolios digitales y se compartió en grupo, lo que permitió revisar, comentar y reforzar lo aprendido entre todos.

5. Resultados

a) Portafolios digitales.

- **Camila Beltrán:**

https://drive.google.com/drive/folders/1RuhYbggyaCuSWSLuvtp0TIOxiZyyUSFGN?usp=drive_link

- **Alexander Guamán:**

https://drive.google.com/drive/folders/1_w1XAa7pq1C0bIIG7wtn3XvE4ZTi9Mq2?usp=drive_link

- **Lady Guamán:**

<https://drive.google.com/drive/folders/1mTlo3we349Y11Isn0G2fURQ2MLU8BjYY?usp=sharing>

- Juan Jumbo:

https://drive.google.com/drive/folders/1f1WAJUOeQp3AMpHOT83idQY9R61PY7Rq?usp=drive_link

b) Tablas de verdad con 2,3 y 4 variables.

1. $(p \vee q) \wedge \sim (\sim p \wedge q)$

p	q	$\sim q$	$p \vee q$	$p \vee \sim q$	$(p \vee q) \wedge \sim (\sim p \wedge q)$
V	V	F	V	V	V
V	F	V	V	V	V
F	V	F	V	F	F
F	F	V	F	V	F

- Clasificación: Contingencia (valores de verdad y falsos).

2. $[(p \wedge q) \vee (p \wedge \sim q)] \vee (r \wedge \sim r)$

p	q	r	$\sim q$	$\sim r$	$p \wedge q$	$p \wedge \sim q$	$(p \wedge q) \vee (p \wedge \sim q)$	$r \wedge \sim r$	$[(p \wedge q) \vee (p \wedge \sim q)] \vee (r \wedge \sim r)$
V	V	V	F	F	V	F	V	F	V
V	V	F	F	V	V	F	V	F	V
V	F	V	V	F	F	V	V	F	V
V	F	F	V	V	F	V	V	F	V
F	V	V	F	F	F	F	F	F	F
F	V	F	F	V	F	F	F	F	F
F	F	V	V	F	F	F	F	F	F
F	F	F	V	V	F	F	F	F	F

- Clasificación: Contingencia (valores de verdad y falsos).

3. $\sim p \vee (q \wedge r)$

p	q	r	$\sim p$	$q \wedge r$	$\sim p \vee (q \wedge r)$
V	V	V	F	V	V
V	V	F	F	F	F
V	F	V	F	F	F
V	F	F	F	F	F
F	V	V	V	V	V
F	V	F	V	F	V
F	F	V	V	F	V
F	F	F	V	F	V

- Clasificación: Contingencia (valores de verdad y falsos).

4. $(p \wedge q) \leftrightarrow (r \vee s)$

p	q	r	s	$p \wedge q$	$r \vee s$	$(p \wedge q) \leftrightarrow (r \vee s)$
---	---	---	---	--------------	------------	---

V	V	V	V	V	V	V
V	V	V	F	V	V	V
V	V	F	V	V	V	V
V	V	F	F	V	F	F
V	F	V	V	F	V	F
V	F	V	F	F	V	F
V	F	F	V	F	V	F
V	F	F	F	F	F	V
F	V	V	V	F	V	F
F	V	V	F	F	V	F
F	V	F	V	F	V	F
F	V	F	F	F	F	V
F	F	V	V	F	V	F
F	F	V	F	F	V	F
F	F	F	V	F	V	F
F	F	F	F	F	F	V

- **Clasificación:** Contingencia (valores de verdad y falsos).

5. $(p \rightarrow q) \wedge (p \vee q)$

p	q	$p \rightarrow q$	$p \vee q$	$(p \rightarrow q) \wedge (p \vee q)$
V	V	F	V	F
V	F	F	V	F
F	V	V	V	V
F	F	F	F	F

- **Clasificación:** Contingencia (valores de verdad y falsos).

c) Simplificación de expresiones lógicas.

1. $\sim (p \rightarrow q) \vee (p \wedge r)$ Ley del condicional
 $\sim (\sim p \vee q) \vee (p \wedge r)$ Ley de Morgan
 $(p \wedge \sim q) \vee (p \wedge r)$ Ley de propiedad distributiva $p \wedge (\sim q \vee r)$
2. $[(p \wedge q) \rightarrow (r \vee \sim r)] \wedge (s \vee \sim s)$ Ley de complemento $(p \wedge q) \rightarrow V \wedge V$ Ley de complemento
 $p \wedge q \rightarrow V$ Ley de condicional
 V
3. $(p \vee q) \wedge (p \vee \sim q)$ Ley de propiedad distributiva $p \vee (q \wedge \sim q)$ Ley de complemento
 $p \vee F$ Ley de ídem potencia p
4. $(p \wedge q) \vee (\sim p \wedge q) \vee (\sim p \wedge s)$ Ley de propiedad distributiva $q \wedge (p \vee \sim p) \vee (\sim p \wedge s)$
Ley de complemento $(q \wedge V) \vee (\sim p \wedge s)$ Ley de complemento
 $q \vee (\sim p \wedge s)$ Ley de propiedad distributiva
 $(q \vee \sim p) \wedge (q \vee s)$
5. $[(p \vee q \vee r) \vee (\sim p \wedge \sim q \wedge \sim r)] \wedge [(q \wedge r) \vee (q \wedge \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$ Ley de Morgan

$[(p \vee q \vee r) \vee \sim(p \vee q \vee r)] \wedge [(q \wedge r) \vee (q \wedge \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$ Ley de complemento

$\vee \wedge [q \wedge (r \vee \sim r) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$ Ley distributiva y complemento

$\vee \wedge [(q \wedge \vee) \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$ Ley de ídem potencia

$\vee \wedge [q \vee (\sim q \wedge r)] \leftrightarrow (q \vee r)$ Ley distributiva

$\vee \wedge [(q \vee \sim q) \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley de complemento

$\vee \wedge [\vee \wedge (q \vee r)] \leftrightarrow (q \vee r)$ Ley de complemento

$\vee \wedge (q \vee r) \leftrightarrow (q \vee r)$ Ley de identidad

$q \vee r \leftrightarrow (q \vee r)$ Ley de bicondicional

$\vee$

d) Lógica computacional: Expresiones booleanas en la programación.

- La lógica proposicional y las expresiones booleanas son conceptos fundamentales dentro de la programación y la lógica
- computacional, ya que facilita la toma de decisiones dentro del software y permite al programador controlar el flujo de su código. En la programación, estas expresiones se aplican en estructuras de control como if - else y while.

If – else	while
Se utilizan para desviar el flujo del programa y funcionan como un interruptor lógico. <pre> 1 #include <stdio.h> 2 int main() { 3 int numero = 4; 4 5 if (numero == 4) { 6 printf("Es el numero cuatro."); 7 } else { 8 printf("No es el numero cuatro."); 9 } 10 11 return 0; 12 }</pre> Si el resultado es verdadero, se ejecuta una ruta de código (if), de lo contrario, se ejecuta la alternativa (else).	Es una estructura de control repetitiva (bucle) que ejecutara un bloque de código mientras la expresión booleana asociada sea verdadera. <pre> 1 #include <stdio.h> 2 3 int main() { 4 int gotas = 0; 5 6 // Mientras el vaso tenga menos de 5 gotas, seguimos llenando 7 while (gotas < 5) { 8 gotas = gotas + 1; 9 printf("Cayó una gota. Total: %d\n", gotas); 10 } 11 12 printf("El vaso está lleno!"); 13 return 0; 14 }</pre> - Si la expresión es una tautología (siempre verdadera), el programa se queda en un bucle infinito. - Si es una contradicción (siempre falsa), el código dentro del bucle nunca se ejecutará. - Lo que se busca es que sea una contingencia (verdadera o falsa), que comience con un valor verdadero y cuando el bucle empiece a trabajar cambie para que la condición se vuelva falsa.

- Además, en programación los conectores lógicos no se escriben con símbolos matemáticos como $\wedge$ o $\vee$, ya que cada lenguaje utiliza palabras reservadas o símbolos especiales.

Conector Lógico	Símbolo matemático	Operador en Python	Operador en C / Java
Conjunción	$\wedge$	And	&&
Disyunción	$\vee$	Or	
Negación	$\sim$	not	!

- El operador "and" se utiliza para verificar que varias condiciones se cumplan al mismo tiempo.
- El operador "or" se usa cuando uno o más de sus operandos son verdaderos.
- El operador "not" se utiliza para invertir el valor lógico de una expresión.

6. Preguntas de Control

1. Explique la diferencia entre una proposición simple y una proposición compuesta. ¿Por qué es importante distinguir entre ambas en el contexto de la programación y diseño de algoritmos? Argumente con ejemplos concretos en lenguajes de programación.

- La principal diferencia entre una proposición simple y una compuesta es que mientras la simple es una oración que expresa una única idea y no contiene conectores lógicos (y, o, no), mientras que la compuesta se forma al combinar dos o más proposiciones simples mediante el uso de conectores lógicos. Su valor de verdad no es directo, sino que depende de los valores de sus componentes y se determina sistemáticamente mediante tablas de verdad para identificar si es una tautología, contradicción o contingencia.
- **Importancia en programación y algoritmos**
 - Es fundamental distinguirlas porque las proposiciones son la base de la lógica computacional. Permiten controlar el flujo de los programas mediante la toma de decisiones en el software.
 - Ayudan a que el programador gestione condiciones en estructuras como if-else y while.
 - Al entender cómo se forman las compuestas, podemos aplicar leyes lógicas (como las de Morgan o distributiva) para simplificar las expresiones y que el código sea más eficiente.
- **Ejemplos:**
 - En la práctica, los lenguajes de programación no usan símbolos matemáticos como $\wedge$ o $\vee$ sino operadores específicos:
 - **Ejemplo de Proposición Simple (if-else):** Se usa como un interruptor lógico; si la condición es verdadera, se ejecuta una ruta, y si es falsa, la alternativa.

```
tiene_entrada = True
es_mayor_edad = False

if tiene_entrada and es_mayor_edad:
    print("¡Puedes pasar!")
else:
    print("Te quedas afuera.") # Saldrá esto porque falta una condición.
```

- **Ejemplo de Proposición Compuesta (and/or):** El operador and (conjunción) sirve para validar que varias condiciones se cumplan al mismo tiempo.

```
tengo_efectivo = False
tengo_tarjeta = True

if tengo_efectivo or tengo_tarjeta:
    print("Pago aceptado.") # Saldrá esto porque la tarjeta salvó el día.
```

- **Ejemplo en Bucles (while):** Lo ideal es que la condición sea una contingencia. Esto permite que el bucle inicie como verdadero, pero que el valor cambie al trabajar para que la condición se vuelva falsa y el programa no se quede en un bucle infinito.

```
boton_silencio_activado = True

if not boton_silencio_activado:
    print("Sonando música...")
else:
    print("Silencio total.") # Saldrá esto porque el 'not' invirtió la lógica.
```

7. Conclusiones

- Como conclusión, hemos logrado identificar y clasificar las proposiciones lógicas y compuestas, aplicando conectores lógicos (negación, conjunción, disyunción, condicional y bicondicional) al formular expresiones lógicas. Así mismo, construimos tablas de verdad para las proposiciones compuestas, determinando si es una tautología, contradicción o contingencia. Aplicando las leyes de deducción lógica e inferencia como una segunda opción para llegar a los resultados que nos dan las tablas de verdad. A su vez mejorando nuestras habilidades de trabajo colaborativo junto al pensamiento crítico y la correcta resolución de problemas.

8. Recomendaciones

- **Priorizar con la jerarquía:** Siempre hay que checar bien qué conector va primero antes de armar la tabla, porque si te saltas una negación o un paréntesis, todo el resultado sale mal.
- **No confundir los símbolos:** Es importante no mezclar los símbolos lógicos ($\wedge$, $\vee$) con los que se usan de verdad en el código (como $\&\&$ o $\|$), porque el lenguaje de programación no reconoce los símbolos matemáticos.
- **Simplificar para ahorrar tiempo:** Antes de hacer tablas de 3 o 4 variables, conviene usar las leyes lógicas para achicar la expresión; así es más difícil equivocarse y terminas más rápido.
- **Revisar los bucles:** Al usar un while, hay que asegurar que la condición sea una contingencia para que el programa no se quede trabado en un bucle infinito.

9. Bibliografía

[1] FasterCapital, "Expresiones booleanas: dominar expresiones booleanas en declaraciones combinadas," *FasterCapital*, 12 de diciembre de 2025. [En línea]. Disponible en: <https://fastercapital.com/es/contenido/Expresionesbooleanas--dominar-expresiones-booleanas-en-declaracionescombinadas.html>.

10. Anexos

- Alexander Guaman

5) $(pq) \rightarrow (rv)$

p	q	r	s	$(pq) \rightarrow (rv)$
V	V	V	V	V
V	V	V	F	V
V	V	F	V	V
V	V	F	F	F
V	F	V	V	F
V	F	V	F	F
V	F	F	V	F
V	F	F	F	V
F	V	V	V	F
F	V	V	F	F
F	V	F	V	F
F	V	F	F	V
F	F	V	V	V
F	F	V	F	V
F	F	F	V	V
F	F	F	F	F

CS Escaneado con CamScanner

Escaneado con CamScanner

• Camila Beltrán

APE 1 - FASE 2

Nombre: Camila Anibal Beltrán Ramírez
 Asignatura: Matemática Discreta
 Códigos: Lógica Matemática

Fecha: Lunes, 05 de Abril de 2021
 Asignatura: Matemática Discreta

1) Construye tabla de verdad para expresiones de 2, 3 y 4 variables.
 2) Resuelve expresiones de simplificación de expresiones lógicas.

1) $(p \vee q) \wedge \neg(p \wedge q)$ 2 Variables

• TABLA

p	q	$p \vee q$	$p \wedge q$	$\neg(p \wedge q)$	$(p \vee q) \wedge \neg(p \wedge q)$
V	V	V	V	F	F
V	F	V	F	V	V
F	V	V	F	V	V
F	F	F	F	V	F

Contingencia

2) $(p \rightarrow q) \wedge (p \vee q)$ 2 Variables

• TABLA

p	q	$p \rightarrow q$	$p \vee q$	$(p \rightarrow q) \wedge (p \vee q)$
V	V	V	V	V
V	F	F	V	F
F	V	V	V	V
F	F	V	F	F

Contingencia

3) $[(p \wedge q) \vee (p \wedge \neg q)] \vee (q \wedge \neg p)$ 3 Variables

• TABLA

p	q	r	$p \wedge q$	$p \wedge \neg q$	$q \wedge \neg p$	$[(p \wedge q) \vee (p \wedge \neg q)] \vee (q \wedge \neg p)$
V	V	V	V	F	F	V
V	V	F	V	F	F	V
V	F	V	F	V	F	V
V	F	F	F	V	F	V
F	V	V	F	F	V	V
F	V	F	F	F	V	V
F	F	V	F	F	F	F
F	F	F	F	F	F	F

Contingencia

• SIMPLIFICACION

$[(p \wedge q) \vee (p \wedge \neg q)] \vee (q \wedge \neg p) \rightarrow$ Ley distributiva.
 $(p \wedge (q \vee \neg q)) \vee (q \wedge \neg p) \rightarrow$ Ley de complemento.
 $p \vee (q \wedge \neg p) \rightarrow$ Ley de Idem potencia.
 $p \vee q$

4) $[(p \wedge q) \rightarrow (r \vee \neg r)] \wedge (s \vee \neg s)$ 4 Variables

• TABLA

p	q	r	s	$p \wedge q$	$r \vee \neg r$	$s \vee \neg s$	$(p \wedge q) \rightarrow (r \vee \neg r)$	$[(p \wedge q) \rightarrow (r \vee \neg r)] \wedge (s \vee \neg s)$
V	V	V	V	V	V	V	V	V
V	V	V	F	V	V	V	V	V
V	V	F	V	V	V	V	V	V
V	V	F	F	V	V	V	V	V
V	F	V	V	F	V	V	V	V
V	F	V	F	F	V	V	V	V
V	F	F	V	F	V	V	V	V
V	F	F	F	F	V	V	V	V
F	V	V	V	V	V	V	V	V
F	V	V	F	V	V	V	V	V
F	V	F	V	V	V	V	V	V
F	V	F	F	V	V	V	V	V
F	F	V	V	F	V	V	V	V
F	F	V	F	F	V	V	V	V
F	F	F	V	F	V	V	V	V
F	F	F	F	F	V	V	V	V

Tautología

• SIMPLIFICACION

$[(p \wedge q) \rightarrow (r \vee \neg r)] \wedge (s \vee \neg s) \rightarrow$ Ley de complemento.
 $[(p \wedge q) \rightarrow V] \wedge V \rightarrow$ Ley de Condicional.
 $V \wedge V$
 $V //$

5) $[(p \vee q) \vee (p \wedge \neg q)] \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r)$ 3 Variables

• Simplificación

$[(p \vee q) \vee (p \wedge \neg q)] \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de Morgan.
 $[(p \vee q) \vee (p \wedge \neg q)] \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de complemento.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley distributiva y complemento.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de Idem potencia.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley distributiva.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de complemento.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de Idem potencia.
 $V \wedge [(q \wedge \neg p) \vee (q \wedge p)] \leftrightarrow (q \vee r) \rightarrow$ Ley de Identidad.
 $(q \vee r) \leftrightarrow (q \vee r) \rightarrow$ Ley de bicondicional.
 $V //$

• TABLA:

p	q	r	p ∨ q	p ∧ q	q ∨ r	(p ∨ q) ∨ (p ∧ q)	(q ∨ r) ∨ (p ∨ q)	(p ∨ q) ∧ (q ∨ r)	[(p ∨ q) ∨ (p ∧ q)] ∧ [(q ∨ r) ∨ (p ∨ q)]	[(p ∨ q) ∨ (p ∧ q)] ∧ [(q ∨ r) ∨ (p ∨ q)]	[(p ∨ q) ∨ (p ∧ q)] ∧ [(q ∨ r) ∨ (p ∨ q)]	[(p ∨ q) ∨ (p ∧ q)] ∧ [(q ∨ r) ∨ (p ∨ q)]
V	V	V	V	V	V	V	V	V	V	V	V	V
V	V	F	V	V	V	V	V	V	V	V	V	V
V	F	V	V	F	V	V	V	V	V	V	V	V
V	F	F	V	F	F	V	V	F	F	F	F	F
F	V	V	V	V	V	V	V	V	V	V	V	V
F	V	F	V	V	V	V	V	V	V	V	V	V
F	F	V	F	F	V	V	V	F	F	F	F	F
F	F	F	F	F	F	F	F	F	F	F	F	F

Tautología

LÓGICA COMPUTACIONAL: Expresiones booleanas en la programación.

La lógica proposicional y las expresiones booleanas son herramientas fundamentales dentro de la programación y la lógica computacional, ya que facilitan la toma de decisiones dentro del software y permite al programador controlar el flujo de su código. En la programación, estas expresiones se aplican en estructuras de control como if-else y while.

Conector Lógico	Símbolo matemático	Operador en Python	Operador en C / Java
Conjunción	$\wedge$	And	& &
Disyunción	$\vee$	Or	
Negación	$\sim$	Not	!

IF - else

Se utilizan para desviar el flujo del programa y funcionan como un interruptor lógico.

EJEMPLO:

```
# incluye stdio.h
int main() {
    int numero = 4;
    if (numero == 4) {
        printf("Es el número cuatro.");
    } else {
        printf("No es el número cuatro.");
    }
    return 0;
}
```

Si el resultado es verdadero, se ejecuta una ruta de código (if), de lo contrario, se ejecuta la alternativa (else).

While

Es una estructura de control repetitiva (bucle) que ejecuta un bloque de código mientras la expresión booleana asociada sea verdadera.

EJEMPLO:

```
# incluye stdio.h
int main() {
    int gotas = 0;
    while (gotas < 5) {
        gotas = gotas + 1;
        printf("Se cayó una gota total: %d\n", gotas);
    }
    printf("¡El vaso está lleno!");
    return 0;
}
```

Si la expresión es una tautología (siempre verdadera), el programa se queda en un bucle infinito.
Si es una contradicción (siempre falsa), el código dentro del bucle nunca se ejecutará.
Lo que se busca es que sea una contingencia (verdadera o falsa), que comience con un valor verdadero y cuando el bucle empiece a trabajar, cambie para que la condición se vuelva falsa.

Además, en programación los conectores lógicos no se escriben con símbolos matemáticos como $\wedge$ o $\vee$, ya que cada lenguaje utiliza palabras reservadas o símbolos especiales.

Conector Lógico	Símbolo matemático	Operador en Python	Operador en C / Java
Conjunción	$\wedge$	And	& &
Disyunción	$\vee$	Or	
Negación	$\sim$	Not	!

El operador "and" se utiliza para verificar que varias condiciones se cumplan al mismo tiempo.
El operador "or" se utiliza cuando uno o más de sus operandos son verdaderos.
El operador "not" se utiliza para invertir el valor lógico de una expresión.

- Lady Guamán:

APEI Fase 2

Nombre: Lady Guaman Fecha: 06-04-2026
 Docente: Mario Cueva Asignatura: Matemáticas Discretas
 Círculo: 1er ciclo "A"

1) Construir tablas de verdad para expresiones de 2, 3 y 4 variables

2) Resolver ejercicios de simplificación de expresiones lógicas

1) $(p \wedge q) \vee (p \wedge r)$
 $p \wedge (q \vee r) = (p \wedge q) \vee (p \wedge r)$

p	q	r	$p \wedge q$	$p \wedge r$	$(p \wedge q) \vee (p \wedge r)$	$p \wedge (q \vee r)$
V	V	V	V	V	V	V
V	V	F	V	F	V	V
V	F	V	F	V	V	V
V	F	F	F	F	F	F
F	V	V	F	V	V	V
F	V	F	F	F	F	F
F	F	V	F	F	F	F
F	F	F	F	F	F	F

$p \wedge q \vee (p \wedge r)$
 $p \wedge (q \vee r) = p \wedge q \vee (p \wedge r)$ Ley Distributiva

2) $p \wedge \neg p$ $p \wedge \neg p \rightarrow$ Ley de complemento

p	$\neg p$	$p \wedge \neg p$
V	F	F
F	V	F

Contradicción

3) $(p \wedge q) \vee (\neg p \wedge r)$ simplificación
 $(p \wedge q) \vee (\neg p \wedge r)$

p	q	r	$p \wedge q$	$\neg p \wedge r$	$(p \wedge q) \vee (\neg p \wedge r)$
V	V	V	V	F	V
V	V	F	V	F	V
V	F	V	F	V	V
V	F	F	F	F	F
F	V	V	F	V	V
F	V	F	F	F	F
F	F	V	F	V	V
F	F	F	F	F	F

Contingencia

4) $(p \vee q) \rightarrow (p \wedge q)$ simplificación
 $(p \vee q) \rightarrow (p \wedge q)$ Ley de complemento

p	q	$p \vee q$	$p \wedge q$	$(p \vee q) \rightarrow (p \wedge q)$
V	V	V	V	V
V	F	V	F	F
F	V	V	F	F
F	F	F	F	V

Tautología

5) $[(p \wedge q) \vee (p \wedge \neg q)] \vee (\neg p \wedge r)$

p	q	r	$p \wedge q$	$p \wedge \neg q$	$\neg p \wedge r$	$[(p \wedge q) \vee (p \wedge \neg q)] \vee (\neg p \wedge r)$
V	V	V	V	F	F	V
V	V	F	V	F	F	V
V	F	V	F	V	F	V
V	F	F	F	V	F	V
F	V	V	F	F	V	V
F	V	F	F	F	F	F
F	F	V	F	F	V	V
F	F	F	F	F	F	F

Contingencia

Simplificación

$[(p \wedge q) \vee (p \wedge \neg q)] \vee (\neg p \wedge r) \rightarrow$ Ley distributiva

$[(p \wedge (q \vee \neg q))] \vee (\neg p \wedge r) \rightarrow$ Ley de complemento

$(p \wedge V) \vee (\neg p \wedge r) \rightarrow$ Ley de idem potencia

$p \vee (\neg p \wedge r) \rightarrow$ Ley de idem potencia

p

Lógica Computacional

Expresiones booleanas en la programación

La lógica proposicional y las expresiones booleanas son conceptos fundamentales dentro de la programación y la lógica computacional, ya que facilitan la toma de decisiones dentro del software y permite al programador controlar el flujo de su código. En la programación estas expresiones se aplican en estructuras de control como if-else y while.

if-else	while
Se utilizan para desviar el flujo del programa y funcionan como un interruptor lógico.	Es una estructura de control repetitiva (bucle) que ejecuta un bloque de código mientras la expresión booleana asociada sea verdadera.
<pre> int main() { int numero = 4; if (numero == 4) { printf("Es el número cuatro"); } else { printf("No es el número cuatro"); } return 0; } </pre>	<pre> #include <stdio.h> int main() { int gates = 0; // Mientras el usuario ingrese menos de 5 puntos, seguimos llenando while (gates < 5) { gates = gates + 1; printf("¡Ganó una gatea!\n"); printf("Total: %d\n", gates); } } </pre>

Si el resultado es tautología, se ejecuta una ruta de código (true), de lo contrario se ejecuta la alternativa (else).

3
 printf("¡Eso es tautología!");
 return 0;

3
 - Si la expresión es una tautología (siempre verdadera), el programa se queda en bucle infinito.
 - Si es una contradicción (siempre falsa), el código dentro del bucle nunca se ejecutará.

Lo que se busca es que sea una contingencia (verdadera o falsa), que comience con un valor verdadero y cuando el bucle empiece a trabajar cambie para que la condición se vuelva falsa.

Además en programación los conectores lógicos no se escriben con símbolos matemáticos como $\wedge$ o $\vee$, ya que cada lenguaje utiliza palabras reservadas o símbolos especiales.

Conector lógico	Símbolo matemático	Operador en Python	Operador en C++/Java
Conjunción	$\wedge$	and	&&
Disyunción	$\vee$	or	
Negación	$\neg$	not	!

- El operador "and" se utiliza para verificar que varias condiciones se cumplan al mismo tiempo.
- El operador "or" se usa cuando uno o más de sus operandos son verdaderos.
- El operador "not" se utiliza para invertir el valor lógico de una expresión.

ACTIVIDAD PRACTICA EXPERIMENTAL 2

NOMBRE: JUAN PABLO JIMBO ORECIANA

2. CONSTRUCCIÓN TABLAS DE VERDAD

TABLA 2 VARIABLES

EXPRESION: $(p \rightarrow q) \wedge (q \rightarrow p)$

P	q	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$
V	V	V	V	V
V	F	F	V	F
F	V	V	F	F
F	F	V	V	V

El resultado es una contingencia

TABLA 3 VARIABLES

EXPRESION: $(p \wedge q) \rightarrow r$

P	q	r	$p \wedge q$	$(p \wedge q) \rightarrow r$
V	V	V	V	V
V	V	F	V	F
V	F	V	F	V
V	F	F	F	V
F	V	V	F	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	V

El resultado es una contingencia

TABLA 4 VARIABLES

EXPRESION: $[(p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r)] \rightarrow (q \vee s)$

P	q	r	s	$p \rightarrow q$	$r \rightarrow s$	$p \vee r$	$[(p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r)] \rightarrow (q \vee s)$
V	V	V	V	V	V	V	V
V	V	V	F	V	F	V	F
V	V	F	V	V	V	V	V
V	V	F	F	V	V	V	V
V	F	V	V	F	V	V	F
V	F	V	F	F	F	V	V
V	F	F	V	V	V	V	V
V	F	F	F	V	V	V	V
F	V	V	V	V	V	V	V
F	V	V	F	V	F	V	F
F	V	F	V	V	V	V	V
F	V	F	F	V	V	V	V
F	F	V	V	V	V	V	V
F	F	V	F	V	F	V	F
F	F	F	V	V	V	V	V
F	F	F	F	V	V	V	V

El resultado es una tautología

3 EJERCICIOS SIMPLIFICACIÓN LÓGICA

EXPRESION: $\neg(p \wedge q) \vee (p \wedge q)$

$\neg p \wedge q \vee (p \wedge q)$ LEY DE MORGAN

$(\neg p \wedge q) \vee p = \neg p \wedge q \vee p \wedge q$ LEY DE ABSORCIÓN

$(p \vee q) \wedge q = p \vee q \wedge q$ LEY DE ABSORCIÓN

$p \vee q \wedge q$

$p \vee q$

EXPRESION: $(p \wedge (p \vee q)) \vee (q \wedge \neg q)$ LEY DE ABSORCIÓN

$p \vee q \wedge q$ LEY DE COMPLEMENTO

$p \vee F$ LEY DE IDENTIDAD

p

EXPRESION: $\neg[(p \wedge q) \rightarrow r] \vee (q \wedge p)$

$\neg[p \vee \neg q] \vee (q \wedge p)$ LEY DE MORGAN

$\neg[p \wedge \neg r] \vee (q \wedge p)$ LEY DE ABSORCIÓN, LEY CONDICIONAL

$\neg[\neg p \vee r] \vee q \wedge p$ LEY DE MORGAN

$p \wedge q \vee q \wedge p$ LEY DE ABSORCIÓN

$q \vee p \wedge p$ LEY DE ABSORCIÓN

p

EXPRESION: $(p \vee q) \wedge \neg(p \wedge q)$

$(p \vee q) \wedge (p \vee \neg q)$ LEY DE MORGAN, DISTRIBUTIVA INVERSA

$p \vee (q \wedge \neg q)$ LEY DE COMPLEMENTO

$p \vee F$ LEY DE IDENTIDAD

p

EXPRESION: $[(p \rightarrow q) \wedge \neg q] \vee [\neg(p \rightarrow q) \vee \neg(p \vee r)]$

$[\neg p \vee q] \wedge \neg q \vee [\neg(\neg p \vee \neg q) \vee \neg(p \vee r)]$ LEY MORGAN, CONDICIONAL

ABSORCIÓN: $[\neg q \wedge p] \vee [\neg p \wedge (r \vee \neg r)]$ LEY DISTRIBUTIVA INVERSA

$\neg q \wedge p \vee \neg p \wedge V$ LEY DE COMPLEMENTO, IDENTIDAD

$\neg q \wedge p \vee \neg p$ LEY DE ABSORCIÓN

$\neg p$

4. COMO LAS EXPRESIONES LÓGICAS SE APLICAN EN

IF - ELSE se usan para determinar alguna condición que se vaya a ejecutar y por ejemplo el **if** solo se ejecuta cuando la expresión booleana resulta en verdadera y en cambio si es falsa se ejecutará el **else**.

while: en este el bucle hecho continúa repitiéndose mientras la condición evaluada siga siendo verdadera pero cuando la condición sea falsa el programa sale del bucle.

OPERADORES LÓGICOS

OPERACION LÓGICA	SÍMBOLO	PYTHON	C/JAVA
CONJUNCIÓN	$\wedge$	and	&&
DISYUNCIÓN	$\vee$	or	
NEGACIÓN	$\sim$	not	!

Algo importante es que los lenguajes de programación también siguen un orden ya que primero evalúan los parentesis luego la negación después el **and** y al último el **or**.

Ejemplo en C

```
#include <stdio.h>

int main() {
    int gotas = 0;
    while (gotas < 5) {
        gotas = gotas + 1;
        printf("Cayo una gota. Total: %d", gotas);
    }
    printf("El vaso está lleno!");
    return 0;
}
```

Ejemplo en C

```
#include <stdio.h>

int main() {
    int numero = 4;
    if (numero == 4) {
        printf("El numero es cuatro");
    }
    else {
        printf("No es el numero cuatro");
    }
    return 0;
}
```